

1. What is Prompt Engineering?

Prompt Engineering is the process of designing clear and specific instructions for an AI model to produce a useful and relevant response.

A good prompt usually provides some combination of:

- **Instruction** — what the AI should do
- **Context** — relevant background information
- **Task requirements** — specific conditions or constraints
- **Role/Perspective** — how the AI should approach the task
- **Output format** — how the response should be structured

Prompt Engineering is useful in applications such as chatbots, AI assistants, coding tools, research systems, customer-support systems, and AI agents.

2. Three Prompt Experiment

Selected Task

Task: Ask an AI model to create a learning plan for a beginner who wants to learn Generative AI.

Prompt 1 — Simple Prompt

Prompt:

Create a learning plan for Generative AI.

AI Response:

A Generative AI learning plan can include:

1. Learn Python basics.
2. Understand machine learning.
3. Learn neural networks.
4. Study Large Language Models.
5. Learn prompt engineering.
6. Work with AI APIs.
7. Build Generative AI projects.

8. Learn about RAG and AI agents.

Observation

The response provides useful information, but it is general. It does not know the learner's experience level, available time, preferred technologies, or specific goal.

Prompt 2 — Prompt With Context and Requirements

Prompt:

Create a 30-day Generative AI learning plan for a beginner who already knows JavaScript, React and basic Python. The learner wants to become an AI application developer. Include daily topics and practical exercises. Focus on LLMs, prompt engineering, AI APIs and small projects.

AI Response:

30-Day Generative AI Learning Plan

Week 1: AI and LLM Fundamentals

- Day 1: Introduction to Generative AI
- Day 2: Understanding LLMs
- Day 3: Tokens and context windows
- Day 4: Prompt Engineering
- Day 5: AI model limitations
- Day 6: Experiment with an AI chatbot
- Day 7: Review and build a small prompt-based application

Week 2: AI APIs

- Learn how to call LLM APIs.
- Understand API keys and requests.
- Build a simple chatbot.

Week 3: AI Application Development

- Connect an LLM with a React frontend.
- Create a backend API.
- Add conversation history.

Week 4: Projects

- Build an AI assistant.
- Build an AI document summarizer.
- Build a small AI-powered application.

Observation

This response is more relevant because the prompt provides the learner's existing skills, goal, duration and required topics.

Prompt 3 — Detailed Prompt With Role, Context and Output Format

Prompt:

Act as a Generative AI mentor and curriculum designer.

Create a practical 30-day learning plan for a beginner-to-intermediate web developer who already knows JavaScript, React, Node.js and basic Python.

The goal is to become an AI application developer capable of building applications using LLM APIs.

Requirements:

- Focus on practical development rather than advanced mathematics.
- Cover Generative AI fundamentals, LLMs, tokens, context windows, prompt engineering, AI APIs, embeddings, RAG and basic AI agents.
- Include one practical exercise for every day.
- Include at least three small projects during the 30 days.
- Use free or low-cost tools where possible.
- Avoid advanced research topics.

Output the result as a table with these columns:

Day | Topic | What to Learn | Practical Exercise | Expected Outcome

Keep each day's plan concise and suitable for approximately 1–2 hours of study.

Observation

This prompt produces a more structured response because it specifies:

- The **role** of the AI
- The **user's background**
- The **learning objective**
- The **required topics**
- The **time limitation**
- The **tools preference**
- The **output format**

4. Improved Final Prompt

After comparing the three prompts, I improved Prompt 3 by making the requirements more precise.

Act as a Generative AI mentor and curriculum designer.

Design a 30-day practical learning plan for a web developer who knows JavaScript, React, Node.js and basic Python and wants to transition into Generative AI application development.

The learner can study **1–2 hours per day**.

Cover:

- Generative AI fundamentals
- LLMs
- Tokens and context windows
- Prompt Engineering
- AI APIs
- Embeddings
- RAG
- Basic AI agents

For every day, provide:

1. Topic
2. Three learning objectives
3. One practical exercise
4. Expected result

Include three progressively larger projects during the 30 days.

Prefer free or low-cost tools and APIs.

Do not include advanced mathematics or research-level concepts.

Present the complete plan in a Markdown table using:

Day | Topic | Learning Objectives | Practical Exercise | Expected Result

Keep each entry concise and realistic for 1–2 hours of work.

5. Five Useful Prompt Collection

Prompt 1 — Learning

Act as an experienced Generative AI mentor. Explain **RAG (Retrieval-Augmented Generation)** to a web developer who knows JavaScript and React but is new to RAG. Explain the concept using a simple real-world example, then explain the basic workflow and list the technologies commonly used to build a RAG application. Keep the explanation beginner-friendly.

Prompt 2 — Summarizing Information

Summarize the following technical article for a beginner software developer. Identify the **main idea, five key points, important technical terms and practical applications**. Do not add information that is not present in the provided text. Use headings and bullet points.

Text:

[Paste article here]

Prompt 3 — Generating Content

Act as a professional LinkedIn content writer. Create a LinkedIn post about completing a Generative AI internship task on **Prompt Engineering Fundamentals**.

Requirements:

- Professional but natural tone
 - Mention what I learned
 - Mention practical prompt experiments
 - Explain why Prompt Engineering is useful for AI applications
 - Keep it under 150 words
 - Use 3–5 relevant hashtags
 - Do not exaggerate my achievements
-

Prompt 4 — Solving a Technical Problem

Act as a senior Python developer. I am building a Streamlit application and my Python file is called `main.py`.

When I run `uv run main.py`, Streamlit displays a warning suggesting that the application should be started using the Streamlit command.

Explain:

1. Why this happens
2. The correct command to run the application
3. The difference between running a normal Python script and a Streamlit application
4. How I can verify that the application is running correctly

Assume I am a beginner and provide the commands step by step.

Prompt 5 — Analyzing Information

Act as a software project analyst. Analyze the following project idea:

AI Trip Planning Application for budget travelers in Pakistan

Analyze it using these categories:

- Target users
- User problem
- Proposed solution
- Key features

- AI features
- Required technologies
- Potential challenges
- Possible improvements

Present the analysis in a table. Clearly separate facts from assumptions.